Learn Python for Automation

Lesson 7: Web Scraping with Requests & BeautifulSoup

Web Scraping with Requests & BeautifulSoup – Study Notes

Key Concepts

1. **HTTP Requests with `requests`** – sending GET/POST calls, handling status codes, headers, and time outs.
2. **Parsing HTML with BeautifulSoup** – creating a soup object, navigating the parse tree, and using selectors (`find`, `find_all`, CSS selectors).
3. **Data Extraction Patterns** – locating tables, lists, or repeated elements; cleaning text; handling relative URLs.
4. **Storing Results** – writing extracted rows to CSV (or JSON) with Python's `csv` module or `pandas`.
5. **Politeness & Legal Considerations** – respecting `robots.txt`, rate limiting, and copyright.

Summary

Web scraping lets you programmatically retrieve information from websites that don't provide a formal API. In a typical beginner workflow, you first **fetch** the raw HTML using the `requests` library, which abstracts the low level details of the HTTP protocol. After confirming a successful response (status code /200), you pass the HTML string to **BeautifulSoup**, a flexible parser that builds a navigable tree of tags.

With the soup object you can **search** for elements by tag name, class, id, or any CSS selector. Common patterns include iterating over rows of a table, extracting links from `<a>` tags, or pulling text from `<p>` elements. Once the desired data is collected, it is cleaned (stripping whitespace, converting types) and then **saved** to a CSV file for later analysis. Throughout the process you should be mindful of the target site's usage policies, throttle your requests, and include a meaningful `User  Agent` header.

Important Points

- **`requests.get(url, headers=..., timeout=...)`**

- Always check `response.status_code`.
- Use a realistic `User-Agent` to avoid being blocked.

- **Creating a soup:**

```python
from bs4 import BeautifulSoup
soup = BeautifulSoup(response.text, "html.parser")   # or "lxml"
```

• **Finding elements:**
- `soup.find(tag, attrs={...})` – first match.
- `soup.find_all(tag, class_="name")` – list of matches.
- `soup.select("css > selector")` – CSS selector syntax.
• **Navigating:**
- `.text` or `.get_text(strip=True)` for clean text.
- `.attrs["href"]` for attribute values.
- `.parent`, `.children`, `.next_sibling`, `.previous_sibling` for tree navigation.
• **CSV Writing (standard library):**
```python
import csv
with open("output.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["Title", "Price"])
    writer.writerows(data_rows)
```
• **Politeness:**
- Honor `robots.txt` (use `urllib.robotparser`).
- Add a short `time.sleep()` between requests (e.g., 1–2 /s).
- Do not hammer a site with parallel requests unless you have permission.

Examples

Example 1 – Scrape a simple product list and save to CSV

```python
import requests, csv, time
from bs4 import BeautifulSoup
BASE_URL = "https://books.toscrape.com/"
response = requests.get(BASE_URL,
                        headers={"User-Agent": "Mozilla/5.0 (study notes)"},
                        timeout=10)
if response.status_code != 200:
    raise SystemExit("Failed to retrieve page")
soup = BeautifulSoup(response.text, "html.parser")
books = soup.select("article.product_pod")
rows = []
for book in books:
    title = book.h3.a["title"]
    price = book.select_one("p.price_color").text.strip()
    rows.append([title, price])
```

Write to CSV

```
with open("books.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["Title", "Price"])
    writer.writerows(rows)
print(f"Saved {len(rows)} books to books.csv")
```

Explanation:

1. **Request** the homepage of a demo site.
2. **Parse** with BeautifulSoup and select each product block.
3. **Extract** the title (from the `<a>` title attribute) and price (text of `<p class="price_color">`).
4. **Save** the list of rows to `books.csv`.

Example 2 – Crawl multiple pages of a table and handle relative URLs

```python
import requests, csv, time
from bs4 import BeautifulSoup
from urllib.parse import urljoin
BASE = "https://example.com/data/"
page = 1
all_rows = []
while True:
    url = f"{BASE}?page={page}"
    r = requests.get(url, headers={"User-Agent": "scraper-demo"}, timeout=10)
    if r.status_code != 200:
        break                    # no more pages
    soup = BeautifulSoup(r.text, "html.parser")
    table = soup.find("table", {"id": "records"})
    if not table:
        break
    for tr in table.tbody.find_all("tr"):
        cols = [td.get_text(strip=True) for td in tr.find_all("td")]
        # Suppose first column contains a relative link to a detail page
        rel_link = tr.find("a")["href"]
        full_link = urljoin(BASE, rel_link)
        cols.append(full_link)
        all_rows.append(cols)
    print(f"Page {page} scraped, {len(all_rows)} rows total")
    page += 1
    time.sleep(1)                # be polite
```

Save

```
with open("records.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["ID", "Name", "Date", "Detail URL"])
    writer.writerows(all_rows)
```

Explanation:

- Loops through paginated results until a non 200 response or missing table.
- Uses `urljoin` to convert relative links (`/detail/123`) into absolute URLs.
- Sleeps 1 /s between pages to avoid over loading the server.

Quick Review

1. **What Python library is used to make HTTP requests, and how do you check that the request succeeded?**
2. **Name two methods for locating elements in a BeautifulSoup object and give a short example of each.**
3. **Why is it important to include a `User-Agent` header and to respect `robots.txt`?**
4. **Show a one line command to write a list of lists `data_rows` to a CSV file called `out.csv`.**
5. **If a link on a page is `/profile/45`, how do you turn it into a full URL using Python?**

Further Reading

- **Advanced parsing:** Using `lxml` parser for speed, handling malformed HTML.
- **Dynamic content:** Selenium, Playwright, or `requests_html` for JavaScript rendered pages.
- **Rate limiting & retries:** `tenacity` library, exponential back off strategies.
- **Data cleaning:** `pandas` for post scrape transformation, handling missing values.
- **Legal & ethical scraping:** GDPR considerations, API alternatives, and terms of service analysis.